DOCKER OPS CHEAT SHEET

Level-3 admin quick reference · Engine 24+ · Compose v2 · the things you forget between incidents

01 / OPERATE
run · inspect · build · store

CONTAINER LIFECYCLE 01

`docker run -d --name web -p 8080:80 nginx` detached + publish

`docker run -it --rm alpine sh` throwaway shell

`docker run -d --restart=unless-stopped app:1.4` survives reboot

`docker ps -a --filter status=exited` what died

`docker ps --filter health=unhealthy` what is failing its healthcheck

`docker ps -s` writable-layer size per container

`docker ps --format '{{.Names}}\t{{.Status}}'` custom columns

`docker inspect -f '{{.RestartCount}}' <c>` is it crash-looping?

`docker stop -t 30 <c>` SIGTERM, then SIGKILL after 30s (default 10)

`docker kill -s HUP <c>` send any signal

`docker rm -f <c>` kill and remove in one

`docker update --memory 2g --cpus 1.5 <c>`
retune limits on a running container

`docker update --restart=unless-stopped $(docker ps -q)`
fix restart policy fleet-wide

`docker start -ai <c>` restart and attach to watch it boot

`docker container prune -f` drop every stopped container

`docker events --since 30m` audit trail: create / die / oom / health

RUN FLAGS THAT EARN THEIR KEEP 02

`-d -it --rm` detached / interactive TTY / self-cleaning

`-p 127.0.0.1:8080:80` publish to loopback only — safer default

`-P` publish every EXPOSEd port on random high ports

`-v vol:/data -v /srv/cfg:/etc/app:ro`
named volume / read-only bind

`--tmpfs /tmp:rw,size=64m` RAM scratch, never touches disk

`-e KEY=val --env-file .env` environment

`-u 1000:1000 -w /app` run as UID:GID / working directory

`--restart no|on-failure:5|always|unless-stopped` restart policy

`--memory 512m --memory-swap 512m` hard cap; equal values disable swap

`--cpus 1.5 --cpuset-cpus 0-3` CFS quota / core pinning

`--pids-limit 200` fork-bomb guard

`--ulimit nofile=65535:65535` file descriptors

`--health-cmd 'curl -fs localhost/health'`
pair with --health-interval 30s --health-retries 3

`--read-only --tmpfs /run` immutable root filesystem

`--cap-drop ALL --cap-add NET_BIND_SERVICE` least privilege

`--security-opt no-new-privileges:true` blocks setuid escalation

`--log-opt max-size=10m --log-opt max-file=3`
stops JSON logs eating the disk

`--add-host host.docker.internal:host-gateway`
reach the host from a Linux container

`--network appnet --network-alias db` user-defined net + DNS alias

VOLUMES & DATA 03

`docker volume create pgdata` named volume, managed by Docker

`docker volume inspect pgdata -f '{{.Mountpoint}}'`
its real host path

`docker volume ls -f dangling=true` orphans nobody mounts any more

`-v /srv/data:/data:Z` SELinux relabel on RHEL/Rocky (:z if shared)

`docker run --rm -v pgdata:/d -v $PWD:/b alpine tar czf /b/pg.tgz -C /d .`
back a volume up to the host

`docker run --rm -v pgdata:/d -v $PWD:/b alpine tar xzf /b/pg.tgz -C /d`
restore it

`docker volume prune -a` deletes unused volumes — the data is gone

- "Permission denied" on a bind mount is almost always a UID/GID mismatch: ls -n the host directory, then --user $(id -u):$(id -g) or chown it. On RHEL, confirm with ausearch -m avc -ts recent before blaming Docker.

INSPECT · LOGS · LIVE DEBUG 04

`docker logs -f --tail 200 --since 15m <c>` tail a time window

`docker logs -t <c> 2>&1 | grep -i error` timestamps + filter

`docker exec -it <c> sh` get a shell (bash may not exist)

`docker exec -u 0 -it <c> sh` get in as root

`docker inspect -f '{{.State.Status}} {{.State.ExitCode}}' <c>`
state and exit code on one line

`docker inspect -f '{{.State.Pid}}' <c>` host PID of PID 1

`docker inspect -f '{{json .State.Health}}' <c> | jq`
why the healthcheck is failing

`docker inspect -f '{{.NetworkSettings.Networks}}' <c>`
nets and IPs

`docker stats --no-stream` one-shot CPU / memory / IO

`docker top <c> -eo pid,ppid,cmd` processes as the host sees them

`docker diff <c>` files changed since the image — config drift

`docker port <c>` the mappings that are really live

`docker cp <c>:/etc/nginx/nginx.conf ./`
pull a file out; works on stopped containers too

`docker attach --sig-proxy=false <c>` watch PID 1, Ctrl-C won't kill it

`docker run -it --rm --pid=container:<c> --network=container:<c> --cap-add SYS_PTRACE nicolaka/netshoot`
a full toolbox inside a distroless container's namespaces

`nsenter -t $(docker inspect -f '{{.State.Pid}}' <c>) -n ss -lntp`
host-side peek into the container's netns

IMAGES · BUILD · REGISTRY 05

`docker build -t app:1.4.2 .` tag real versions, never only latest

`docker build --pull --no-cache -t app:1.4.2 .`
fresh base, full rebuild

`docker build --target prod --build-arg VER=1.4.2 .`
multi-stage target

`docker build --progress=plain .` see every layer and cache miss

`docker buildx build --platform linux/amd64,linux/arm64 -t reg/app:1.4.2 --push .`
multi-arch image in one shot

`docker history --no-trunc app:1.4.2` find the fat layer

`docker image inspect -f '{{.Config.Entrypoint}} {{.Config.Cmd}}' app:1.4.2`
what the container will actually run

`docker pull app@sha256:9f2b…` pin by digest — immutable

`docker save app:1.4.2 | gzip > app.tgz` air-gap export

`gunzip -c app.tgz | docker load` import on the target host

`docker images -f dangling=true -q | xargs -r docker rmi`
drop untagged

`docker image prune -a --filter until=168h` unused for over 7 days

`docker scout cves app:1.4.2` CVE report (or: trivy image app:1.4.2)

DOCKERFILE RULES OF THUMB 06

- Order layers stable → volatile: base, packages, dependency manifest, then source. Reordering alone buys most of your build cache back.
- One RUN chained with && that also deletes /var/lib/apt/lists/* — removing files in a later layer never shrinks the image.
- COPY, not ADD. ADD only when you want a remote tarball auto-extracted.
- Multi-stage: FROM golang AS build … then FROM distroless with COPY --from=build. Compilers and credentials stay out of production.
- Write .dockerignore first (.git, node_modules, tests): smaller context, faster builds, fewer secrets baked in by accident.
- Exec form: ENTRYPOINT ["app"]. Shell form makes PID 1 /bin/sh, SIGTERM never reaches the app, and every stop takes the full 10 seconds.
- USER 10001 before CMD. EXPOSE is documentation — it publishes nothing.
- HEALTHCHECK belongs in the image so every runtime inherits it.

NETWORKING 07

`docker network create appnet`
user-defined bridge: containers resolve each other by name

`docker network create --subnet 172.28.0.0/16 --gateway 172.28.0.1 appnet`
fixed range when corporate subnets collide

`docker network inspect appnet` members, IPs, driver

`docker network connect appnet <c>` attach a running container

`docker run --network host nginx` host netns, no -p, Linux only

`docker run --rm --network appnet nicolaka/netshoot`
dig, tcpdump, ss and curl from inside the same network

`docker exec <c> getent hosts db` test the embedded DNS at 127.0.0.11

`iptables -t nat -nL DOCKER` the NAT rules behind published ports

- The default bridge has no name resolution. Creating a user-defined network fixes most "the app can't find the database" tickets outright.
- -p 8080:80 writes DNAT rules that bypass UFW and firewalld — the port is open to the LAN even when the firewall says it is closed. Publish to 127.0.0.1: and put a reverse proxy in front.

COMPOSE V2 08

`docker compose up -d` v2 is a plugin — no hyphen

`docker compose up -d --build --force-recreate` rebuild and replace

`docker compose down -v --remove-orphans`
-v also destroys named volumes

`docker compose logs -f --tail 100 svc` one service, live

`docker compose exec svc sh` shell into a service

`docker compose pull && docker compose up -d`
roll forward to new images

`docker compose config` render the merged file — validate before deploy

`docker compose -f base.yml -f prod.yml -p acme up -d`
layered overrides + explicit project name

`docker compose run --rm svc rake db:migrate`
one-off task, then clean up

`docker compose up -d --scale worker=4` more workers

`docker compose --profile debug up -d` opt-in services

compose.yaml — the shape of a production service

```
services:
  api:
    image: reg.corp/api:1.4.2
    restart: unless-stopped
    ports: ["127.0.0.1:8080:8080"]
    depends_on: { db: { condition: service_healthy } }
    healthcheck:
      test: ["CMD","curl","-fs","localhost:8080/health"]
      interval: 30s
    deploy: { resources: { limits: { memory: 512M } } }
  db:
    image: postgres:16
    volumes: [pgdata:/var/lib/postgresql/data]
volumes: { pgdata: }
```

DISK, PRUNE & DAEMON 09

`docker system df -v` full breakdown before you delete anything

`docker system prune` stopped ctrs, unused nets, dangling images, cache

`docker system prune -a --volumes` nuclear — takes volume data with it

`docker builder prune --filter until=72h`
BuildKit cache is usually the hidden hog

`du -sh /var/lib/docker/*` where the disk really went

`journalctl -u docker -f --since '10 min ago'` daemon-side errors

`systemctl reload docker` apply daemon.json without killing containers

/etc/docker/daemon.json

```
{
  "log-driver": "json-file",
  "log-opts": { "max-size": "10m", "max-file": "3" },
  "live-restore": true,
  "default-address-pools": [{"base":"172.30.0.0/16","size":24}]
}
```

- /var/lib/docker images, volumes, container logs · /etc/systemd/system/docker.service.d/ proxy env · ~/.docker/config.json registry auth · /etc/docker/certs.d/<registry>/ca.crt private CA

TROUBLESHOOTING PLAYBOOK 10

Exits instantly docker logs, then docker inspect -f '{{.State.ExitCode}}'. Usual cause: the process daemonised or simply finished — PID 1 has to stay in the foreground.

Exit 137 check {{.State.OOMKilled}} and dmesg -T | grep -i oom. Raise --memory or fix the leak; 137 is also a plain docker kill.

Port already allocated ss -lntp | grep :8080 — another container or a host service owns it.

No space left docker system df -v first, then df -i for inodes. Culprits: unrotated JSON logs, build cache, orphaned volumes.

DNS fails inside docker run --rm --network appnet busybox nslookup db. Check the user-defined network, then /etc/resolv.conf inside, then the daemon's dns setting.

Pull: x509 / 401 put the corporate CA in /etc/docker/certs.d/<reg>/ca.crt, the proxy in a systemd drop-in, then docker login.

toomanyrequests Docker Hub rate limit — authenticate, or point registry-mirrors at your own cache.

Permission denied UID/GID mismatch or SELinux. ls -n the host path, add :Z, or run --user $(id -u):$(id -g).

Won't stop the app ignores SIGTERM, usually a shell-form ENTRYPOINT. docker kill now; fix the exec form later.

Slow after months thousands of dead containers and networks. Prune, then check whether log rotation was ever configured.

EXIT CODES 11

0	clean exit
1	application error — read the logs
125	the docker run command itself failed (bad flag)
126	command found but not executable — missing +x
127	command not found — wrong path, wrong base image, missing libc
128+n	killed by signal n
137	SIGKILL — OOM-killed, or docker kill
139	SIGSEGV — segfault
143	SIGTERM — an ordinary docker stop

HARDENING 12

- Never mount /var/run/docker.sock into a container you do not fully trust. It is root on the host, full stop.
- Membership of the docker group is equivalent to root. Audit it the way you audit sudoers.
- Four flags remove most of the risk: --user, --read-only, --cap-drop ALL, --security-opt no-new-privileges:true.
- Secrets never in ENV or ARG — docker history exposes ARG and docker inspect exposes ENV. Mount files or use Compose secrets.
- Pin base images by digest and rebuild on a schedule; a floating tag is not a patching strategy.
- Rootless mode or userns-remap on any host shared between teams.

POWER ONE-LINERS 13

`docker ps -aq -f status=exited | xargs -r docker rm`

`docker images --format '{{.Size}}\t{{.Repository}}:{{.Tag}}' | sort -hr | head`
biggest images first

`docker ps -q | xargs docker inspect -f '{{.Name}} {{.State.Pid}} {{.State.Status}}'`
fleet snapshot

`docker stats --no-stream --format 'table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}'`
tidy resource table

`docker exec -i db psql -U app < dump.sql` pipe a host file straight in

`docker run --rm -v $PWD:/w -w /w alpine sh -c '…'`
borrow a container as a throwaway tool

`watch -n5 'docker ps --format "{{.Names}} {{.Status}}"'`
poor man's dashboard

BEFORE YOU PRUNE
docker system prune -a --volumes on a production host has destroyed more data than most outages. Run docker system df -v and docker volume ls first, and never pass --volumes unless you know exactly what is mounted.